

OWASP Top 10

Mishandling of Exceptional Conditions

BIG school

BIGSEIO

GENERAMOS
NEGOCIO

Mishandling of Exceptional Conditions

BIG school

Índice

Introducción

¿Qué significa esta vulnerabilidad?

¿Por qué ocurre esta vulnerabilidad?

Ejemplos reales y típicos

¿Qué puede hacer un atacante si explota esto?

¿Cómo detectarlo?

¿Cómo prevenirlo?

¿Cómo mitigar si ya estás afectado?

Checklist rápido

Conclusiones



Mishandling of Exceptional Conditions

BIG school

Introducción

- Condiciones excepcionales: errores, timeouts, respuestas inesperadas
- No es un bug funcional: es cómo el sistema reacciona ante lo inesperado
- Si falla mal: fuga de información, comportamientos inseguros o DoS
- Objetivo: diseñar aplicaciones que fallen de forma segura y controlada

¿Qué significa esta vulnerabilidad?

Manejo inseguro de excepciones, errores o condiciones límite

- Stack traces, queries o credenciales expuestas en respuestas de error
- Timeouts/retries mal configurados que agotan recursos
- Respuestas externas no validadas antes de procesar
- Condiciones de carrera (TOCTOU) entre validación y uso

¿Por qué ocurre esta vulnerabilidad?

- Desarrollo centrado en el "camino feliz", errores como afterthought
- Excepciones no controladas que propagan información sensible
- Llamadas externas sin timeout, retry sin límite o validación
- Asumir que APIs de terceros siempre devuelven datos válidos
- Fallbacks que devuelven datos por defecto inseguros

Ejemplos reales y típicos

- Stack trace en respuesta HTTP: revela query, ruta o credenciales
- Timeout infinito en llamada externa: agota hilos o memoria (DoS)
- JSON de tercero sin validar: `data.user.email` sin verificar existencia
- Condición de carrera: validar saldo y debitar en operaciones no atómicas
- Fallback inseguro: error en perfil devuelve objeto "guest" con permisos internos

¿Qué puede hacer un atacante si explota esto?

- Fuga de información sensible: stack traces, queries, configuración interna
- Denegación de servicio: agotar recursos mediante retries o timeouts mal configurados
- Manipulación de lógica: explotar TOCTOU para duplicar acciones o saltar validaciones
- Inyección de datos maliciosos: aprovechar respuestas no validadas en flujos posteriores
- Evasión de controles: forzar errores para activar fallbacks inseguros o bypass de auth

¿Cómo detectarlo?

- **Code review:** bloques catch vacíos, console.log con datos sensibles, respuestas con excepciones
- **Testing:** forzar errores, payloads malformados, timeouts simulados, peticiones concurrentes
- **Producción:** alertas por picos de 5xx, timeouts o excepciones no controladas
- **Señales:** stack traces en respuestas, inconsistencias tras concurrencia, agotamiento de recursos

¿Cómo prevenirlo?

- Fail-safe por defecto: ante error, denegar o devolver mensaje genérico, nunca detalles internos
- Manejo estructurado de excepciones: capturar, loguear seguro en backend, mensaje genérico al cliente
- Timeouts y retries configurados: límites claros + circuit breakers para evitar cascadas de fallo
- Validación de respuestas externas: esquema, tipos y valores antes de usar datos de terceros
- Transaccionalidad/atomicidad: usar transacciones o locks para operaciones críticas (evitar TOCTOU)
- Fallbacks seguros: valores por defecto que no expongan datos sensibles ni privilegios
- Pruebas de resiliencia: chaos testing para validar manejo seguro de fallos

¿Cómo mitigar si ya estás afectado?

- Identificar puntos críticos: endpoints que exponen errores, llamadas sin timeout, lógica con TOCTOU
- Parchear exposición inmediata: ocultar stack traces, validar respuestas, añadir timeouts mínimos
- Implementar manejo global de errores: middleware que capture excepciones y devuelva respuestas genéricas
- Añadir atomicidad: transacciones o locks para operaciones críticas
- Configurar circuit breakers y límites de retry: prevenir agotamiento por fallos en cascada
- Auditar logs: verificar que errores se registren sin exponer datos sensibles

Checklist rápido

- ✓ ¿Excepciones capturadas y logueadas sin exponer stack traces al cliente?
- ✓ ¿Llamadas externas con timeout, retry con límite y validación de respuesta?
- ✓ ¿Operaciones críticas usan transacciones/locks para evitar condiciones de carrera?
- ✓ ¿Fallbacks devuelven valores seguros sin datos sensibles o privilegios extra?
- ✓ ¿Errores se registran con contexto en backend pero mensajes genéricos en frontend?
- ✓ ¿Se prueban escenarios de fallo (timeouts, concurrencia, respuestas malformadas)?
- ✓ ¿Hay alertas para picos de 5xx, timeouts o excepciones no controladas?

Conclusiones

1. El código debe funcionar bien y fallar seguro cuando algo sale mal
2. Fail-safe, validación de respuestas y atomicidad son pilares no negociables
3. Logging seguro de errores: contexto técnico en backend, genérico en frontend
4. Como desarrollador: capturar excepciones, validar respuestas y diseñar fallbacks seguros
5. Pregunta clave: "Si esto falla ahora, ¿qué verá el usuario y qué registrará el sistema?"



¡MUCHAS GRACIAS!

BIG school

BIGSEIO

**GENERAMOS
NEGOCIO**